ARCHITECTURE BRIEF · APRIL 2026

Feature engineering at scale. Fabric × Azure ML.

One physical store. Two logical views. Three layers of responsibility. The defensible production pattern for ML platforms running on Microsoft Fabric.

FREDGIS · GITHUB.COM/FREDGIS/FABRIC-FOUNDRY-KB/

01

The Problem

MICROSOFT FABRIC DOES NOT SHIP A NATIVE FEATURE STORE. SO HOW DO WE RUN FEATURE ENGINEERING AT INDUSTRIAL SCALE ON FABRIC WITHOUT INVENTING A ONE-OFF?

What is a feature store?

A **feature store** is the contract between data engineering and ML serving. It guarantees that a feature computed for training is the **same** feature computed for inference.

It owns four things data alone cannot:

- **Definition** — versioned, immutable, named transformations
- **Lineage** — source-to-prediction graph
- **Materialization** — offline (training) and online (sub-ms serving)
- **Point-in-time correctness** — no leakage

×3 platforms it has to interface — Fabric, Azure ML, ADLS Gen2

0 native feature store in Fabric today

1 defensible production pattern

A Lakehouse table is not a feature store

A LAKEHOUSE DELTA TABLE GIVES YOU

- Schema, ACID, time travel
- Spark and SQL access
- Lineage in Purview
- Sharing via OneLake

A LAKEHOUSE DELTA TABLE DOES NOT GIVE YOU

- Versioned, immutable feature definitions
- A feature retrieval spec embedded in the model
- Online serving in sub-millisecond
- Point-in-time joins primitive
- A registry of which model uses which feature

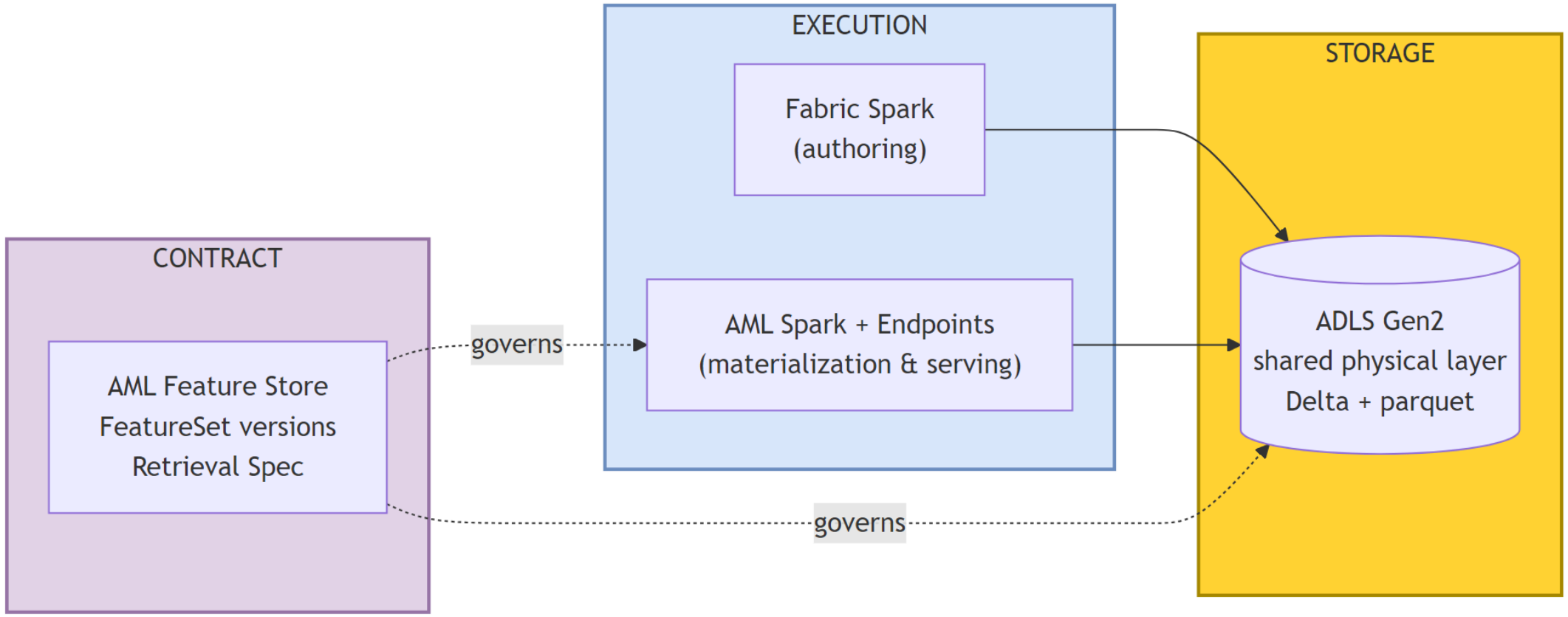
> The Lakehouse table is the **raw material**. The feature store is the **finished product** with a part number on it.

02

The Fundamental Insight

STORAGE ≠ CONTRACT ≠ EXECUTION. THREE LAYERS, THREE RESPONSIBILITIES, THREE AZURE COMPONENTS — DESIGNED TO NEVER OVERLAP.

Three layers, three roles



> **Storage** is one product. **Contract** is another. **Execution** is a third. The day someone tries to merge two of them, push back: storage in Fabric, contract in AML, execution split — that is the rule.

Why Fabric and Azure ML are complementary

What you need	Fabric provides	Azure ML provides
Lakehouse storage on OneLake	**Native (GA)**	None
Spark for feature engineering	**Native (GA)**	Yes (managed clusters)
Feature store + versioning + retrieval spec	None	**Native (GA)**
Online sub-millisecond serving	None	**Native (GA)**
Managed online & batch endpoints	Limited (preview)	**Native (GA)**
Model registry + lineage	Limited	**Native (GA)**
OneLake datastore (preview)	n/a	Public preview

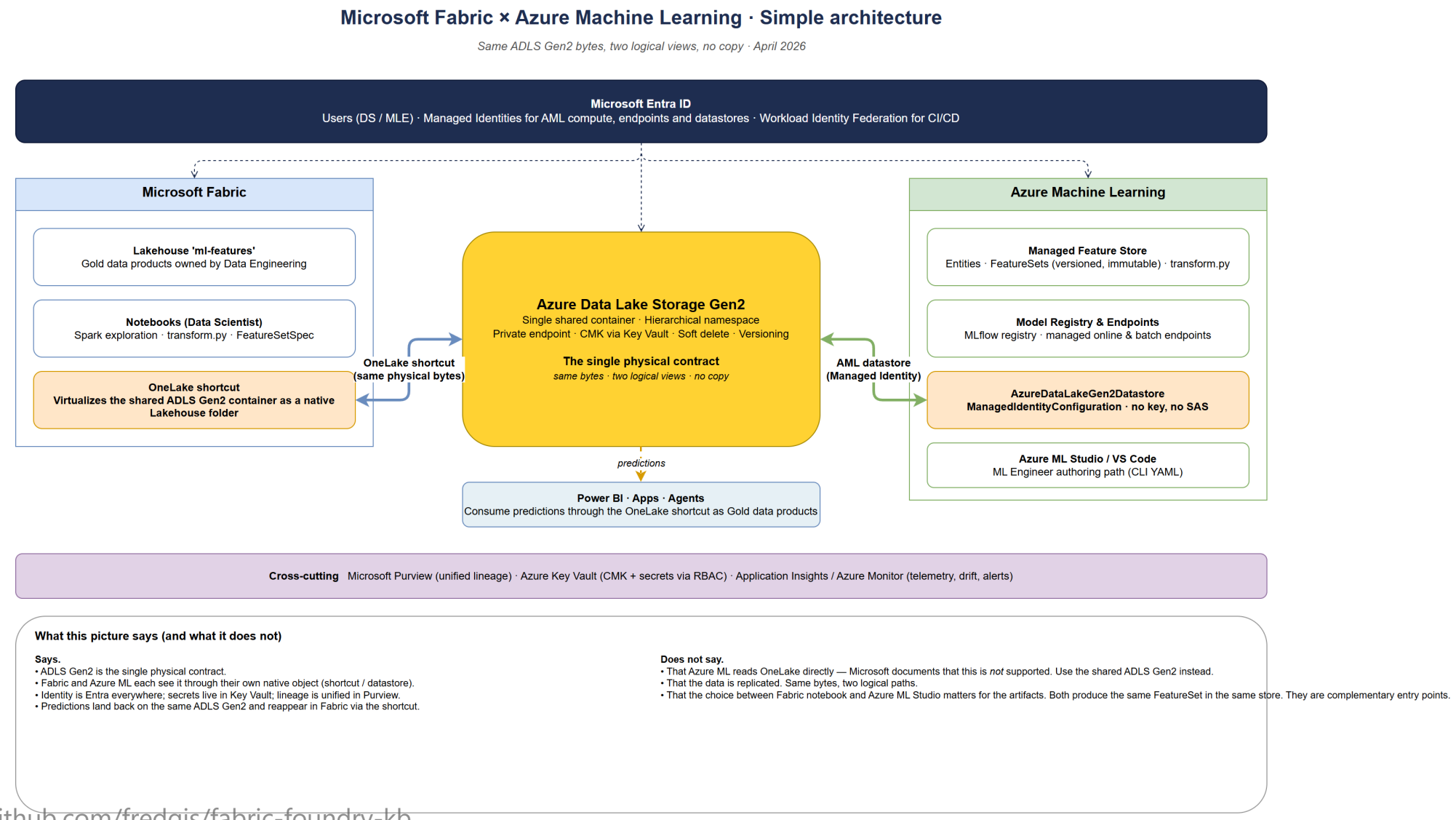
> **They are not redundant.** They are two products built for two adjacent problems. The integration is what makes them an industrial platform.

03

The Recommended Architecture

ONE ADLS GEN2 ACCOUNT PER ENVIRONMENT. TWO LOGICAL VIEWS. ZERO DUPLICATION. THE DEFENSIBLE DEFAULT FOR PRODUCTION TODAY.

Reference architecture — simple view

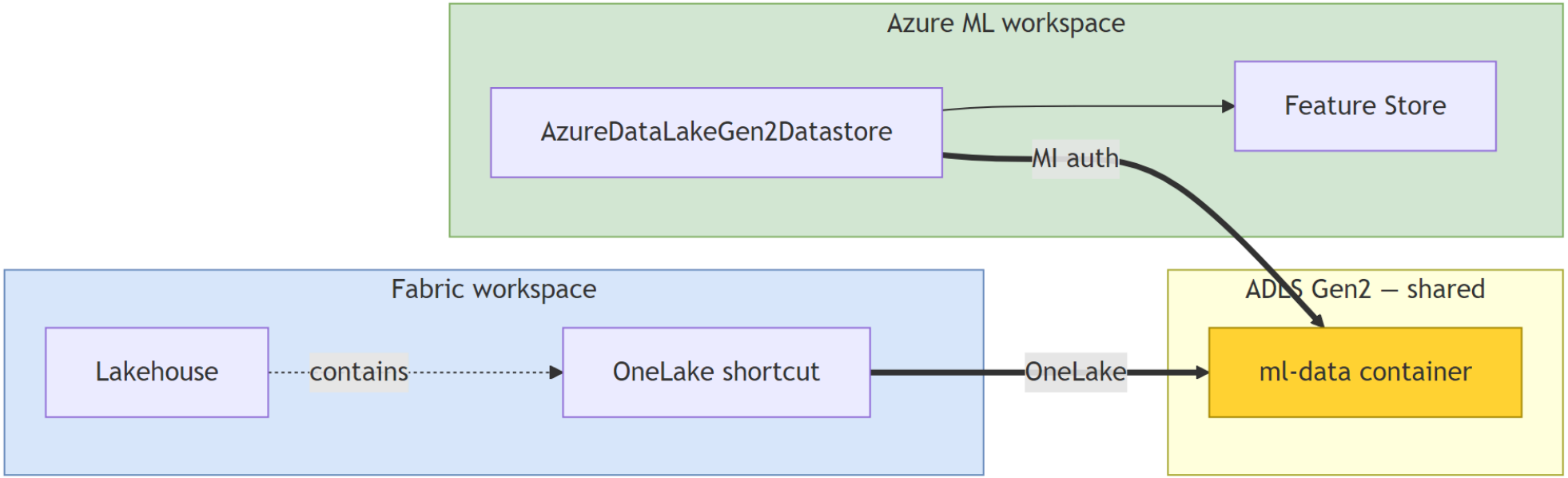


What this picture says (and what it does not)

Says.
- ADLS Gen2 is the single physical contract.
- Fabric and Azure ML each see it through their own native object (shortcut / datastore).
- Identity is Entra everywhere; secrets live in Key Vault; lineage is unified in Purview.
- Predictions land back on the same ADLS Gen2 and reappear in Fabric via the shortcut.

Does not say.
- That Azure ML reads OneLake directly — Microsoft documents that this is *not* supported. Use the shared ADLS Gen2 instead.
- That the data is replicated. Same bytes, two logical paths.
- That the choice between Fabric notebook and Azure ML Studio matters for the artifacts. Both produce the same FeatureSet in the same store. They are complementary entry points.

The contract — ADLS Gen2 + OneLake shortcut



> "Azure Machine Learning can't directly access data stored in Fabric OneLake, but you can configure a OneLake shortcut and an Azure Machine Learning datastore to both access the same Azure Data Lake storage account." — **Microsoft Learn**, `how-to-use-batch-fabric`

Recommended target architecture (single page)

Domain	Recommendation
Storage	One ADLS Gen2 per environment, OneLake shortcut on Fabric, datastore on AML. **No blob versioning** on Delta containers — Delta `_delta_log` owns history.
Authoring	Path A (Fabric notebook) **and** Path B (Studio / VS Code + CLI YAML) coexist. Same artifacts in AML.
Lifecycle	AML Feature Store is the single source of truth. A Lakehouse table is **not** a feature set until it is registered.
Serving	Offline by default. Online (Managed Redis) only if a sync endpoint needs it.
Orchestration	AML pipelines or ADF for SLA-bound batches. Fabric Data Factory AML activity (preview) for non-critical only.
Identity	Managed Identity everywhere. Entra for humans. Federated workload identity for CI/CD. **No SAS, no keys, no long-lived secrets.**
Network	Private endpoints on storage, AML, online store. Trusted Workspace Access for Fabric. Public access OFF.
Governance	Purview ingests both lineages. Sensitivity labels propagate. Each model embeds its retrieval spec.

04

Two Authoring Paths

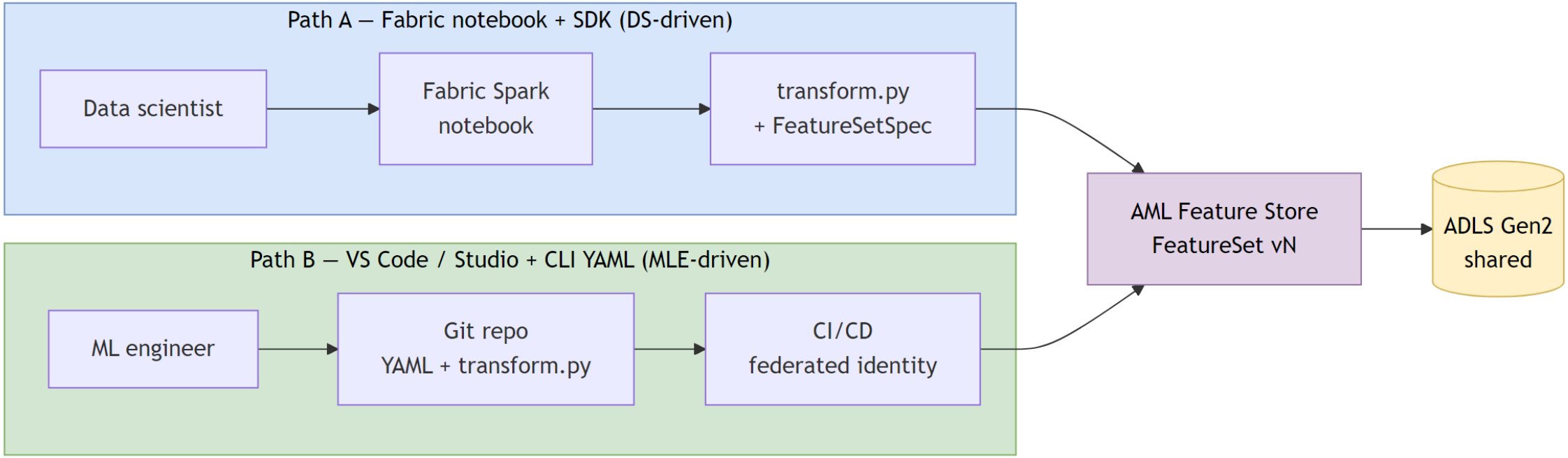
DS-DRIVEN FROM FABRIC, MLE-DRIVEN FROM CI. SAME ARTIFACTS. SAME STORAGE. DIFFERENT HUMANS, DIFFERENT ERGONOMICS.

Two personas — Data Scientist vs ML Engineer

Axis	Data Scientist (Path A)	ML Engineer (Path B)
Question they ask	"Which signal predicts this outcome?"	"How does this run 24/7 without breaking?"
Primary output	Notebook + analysis + proven model	Pipeline + endpoint + monitoring
Typical tools	Fabric notebook, pandas, scikit-learn, MLflow tracking	VS Code, Git, CI/CD, CLI YAML, Docker, MLflow registry
Strong skills	Stats, feature engineering, business storytelling	Software eng., DevOps, observability, FinOps
Success metric	AUC / RMSE on hold-out	p95 latency · error rate · cost-per-1k · feature freshness
Iteration cadence	Hours – days, frozen dataset	Weeks – months, system that must stay up
Owns when model drifts	"Why is performance dropping?"	Alerts, rollback, retraining pipeline

> **MLOps engineer** is one notch further: owns the shared platform tooling (registry, feature store, training infra) that every MLE consumes. Small team = same person. Mature platform = three distinct hats.

Path A vs Path B — both end up in the same place



Decision: which path for this feature set?

Criterion	Path A — Fabric notebook	Path B — VS Code + CLI YAML
Author is a data scientist	✅	
Author is an ML engineer		✅
Feature set is exploratory / Spark-heavy	✅	
Feature set is industrialized / CI-first		✅
Heavy use of Fabric semantic models / mirrored data	✅	
Strong CI/CD discipline required		✅
Both paths in the same team	✅	✅

> Both paths in the same organization is **not** a smell — it is the realistic outcome.

05

Materialization, Streaming, Serving

WHERE THE BYTES PHYSICALLY LIVE, AND HOW OFTEN THEY GET REWRITTEN.
THIS IS WHERE THE BILL IS DECIDED.

Offline vs online store

OFFLINE

ADLS Gen2 parquet

Partitioned by index + timestamp. Serves training, batch inference, exploration, backfills. Latency: seconds (Spark scan).

ONLINE

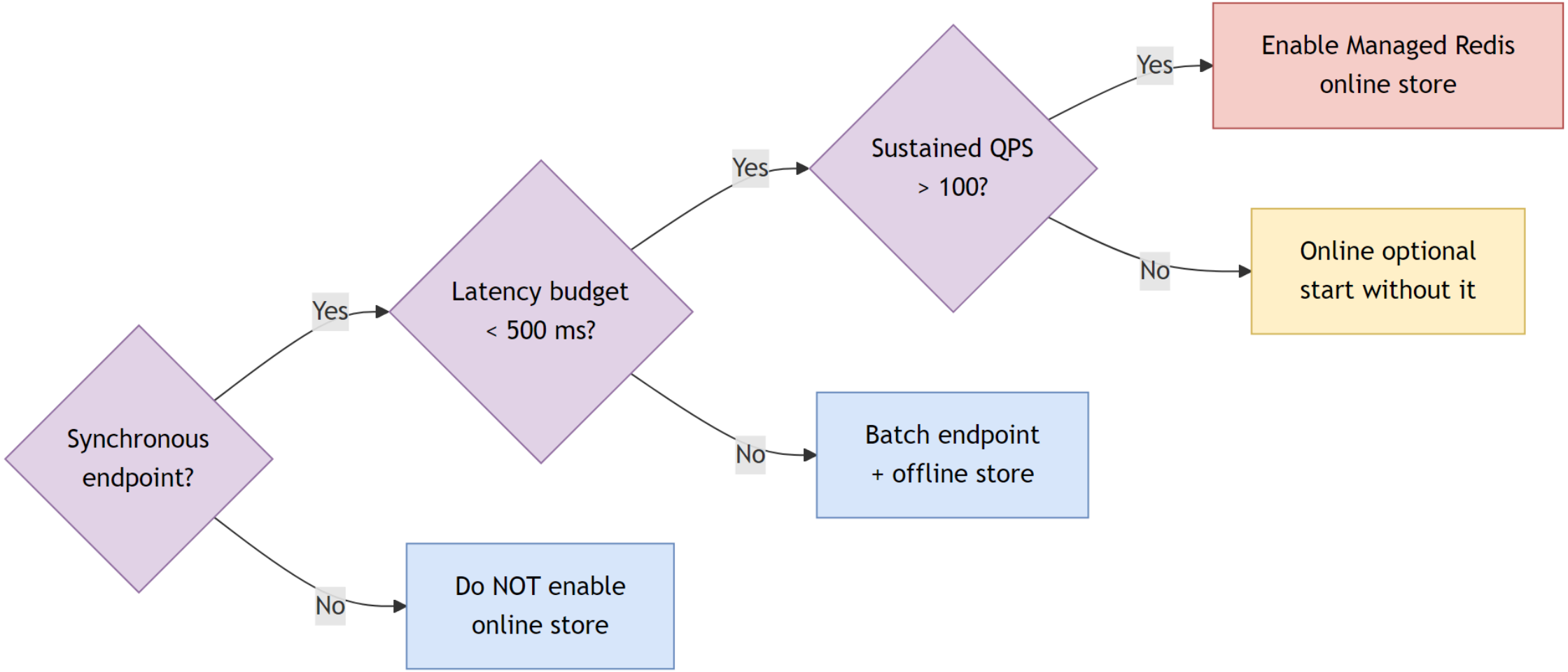
Azure Managed Redis

Sub-millisecond key-value. Serves real-time inference. **Always-on cost** — enable only when a sync endpoint actually needs it.

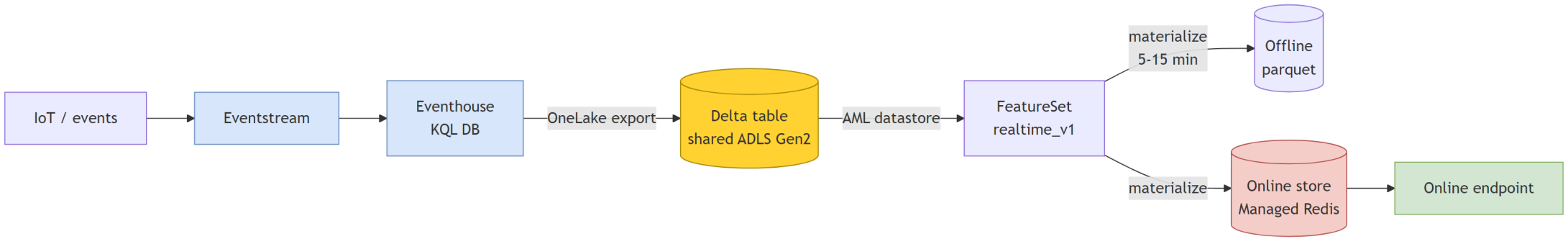
COST LEVERS

- **Frequency** — every minute is 60× the cost of every hour. Align with the model's actual decision cadence.
- **Compute size** — start small (`Standard_E4s_v3` × 2). Scale only when SLA misses.
- **Lookback window** — `source_lookback.days` should reflect what you actually need.

Decision tree — enable the online store?



Real-time / streaming features



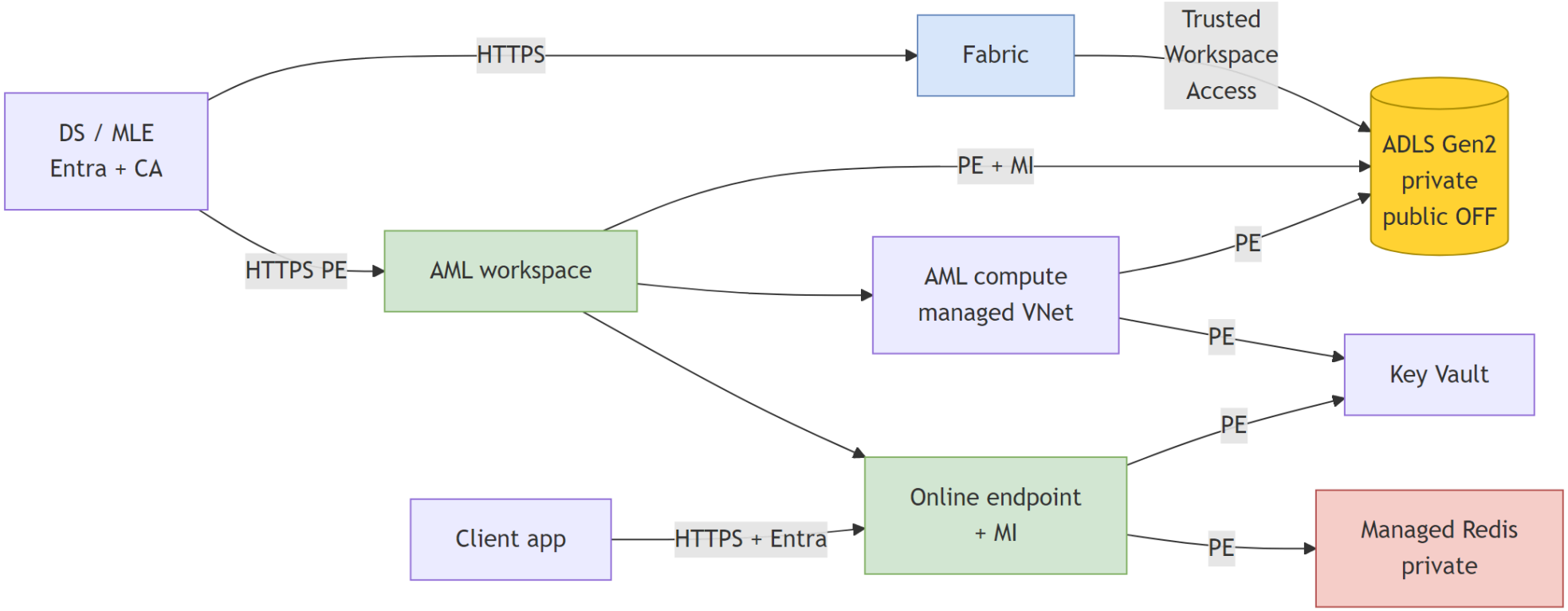
> Fabric **Eventstream → Eventhouse → Delta on shared ADLS → AML FeatureSet** is the production-safe path today. Same storage contract as the batch flow. Pattern alternative: direct push to the online store when sub-second latency is non-negotiable.

06

Governance

IDENTITY, NETWORK, LINEAGE, AUDIT. WHERE THE PLATFORM EITHER EARNS "PRODUCTION-GRADE" OR QUIETLY FAILS ITS FIRST AUDIT.

Minimum-viable production network flow



Identity rules

01 Managed Identity for everything that runs.

AML compute, endpoints, datastore. `ManagedIdentityConfiguration`. No account keys.

02 Entra ID for every human.

Conditional Access enforced on Fabric and on Azure ML. No service principal with stored secret unless absolutely required.

03 Federated workload identity for CI/CD.

GitHub OIDC → Entra. No long-lived secret in pipeline variables.

04 RBAC on Key Vault, not access policies.

`Key Vault Secrets User`, `Key Vault Crypto Service Encryption User`.

Data contracts — the missing validation gate

THE GOLD TABLE SILENTLY DEGRADES. THE FEATURESET KEEPS MATERIALIZING ON NOISE. EVERY DOWNSTREAM MODEL IS WRONG, AND THE AUDIT TRAIL SAYS EVERYTHING IS FINE.

Check	Example	Tooling
Schema	`customer_id` is `string`, non-nullable	Fabric DQ rules / Great Expectations
Freshness	`MAX(last_event_ts) ≥ now() − 24h`	KQL or SQL probe
Cardinality	`COUNT(DISTINCT customer_id)` within ±10 % of 7-day rolling avg	Great Expectations
Null ratio	Per column, null ratio within `baseline ± 5 pp`	Great Expectations
Distribution	KS-statistic vs previous run below threshold	Great Expectations / AML Data Drift

> Validation runs **before** the AML Feature Store materialization. A failed contract halts materialization, alerts the Gold owner, and the FeatureSet keeps serving the previous valid version.

07

FinOps

TWO BILLING UNITS. THREE RULES. ONE WAY TO KEEP THE BILL EXPLAINABLE.

Two platforms, two billing units

Platform	Unit	Granularity	Cost owner
Fabric	Capacity Units (F-SKU or PPU)	Per-second on a shared capacity	Business domain owning the Lakehouse
Azure ML	Compute hours, per-job (batch), per-instance-hour (online), per-tier-hour (online store)	Per-second on dedicated clusters	AML workspace cost center; chargeback to model owner if dominant
Shared infra	ADLS Gen2, Key Vault, Purview, App Insights, private endpoints	Per-GB / per-asset / hour-based	Platform team budget

THREE CHARGEBACK RULES

1. **Author features close to the cheaper compute** — Fabric for exploration when the capacity has headroom; AML for production materialization (right-sizable per FeatureSet).
2. **Tag every AML compute target** with `cost_center` / `model_name` / `feature_set`. Same on Fabric items.
3. **Cap the Fabric capacity for the ML workspace** — a runaway Spark notebook can saturate a capacity shared with Power BI.

Cost map

Component	Driver	Lever
ADLS Gen2	GB × redundancy × tier	Lifecycle policy → cool/archive. Soft delete sized.
AML materialization	Cluster size × runtime × frequency	Right-frequency. Right-size. Spot for non-critical.
AML training	GPU hours × cluster × experiments	Quotas per DS. Auto-shutdown idle clusters.
AML online endpoint	vCPU/RAM × hours × instances	Realistic min/max. Serverless for spiky low-volume.
Online store (Redis)	Tier × shards × hours (always-on)	**Enable only when an online endpoint exists.**
AML batch endpoint	Per-job compute hours	No standing cost. Cheapest for non-real-time scoring.
Network	PE per hour + cross-region egress	One PE per service per env. Same region for ADLS/AML/Fabric.

08

Operations & Decision

SLOS, ALERTS, LIFECYCLE. AND THE QUESTION: DO I REALLY NEED A FEATURE STORE?

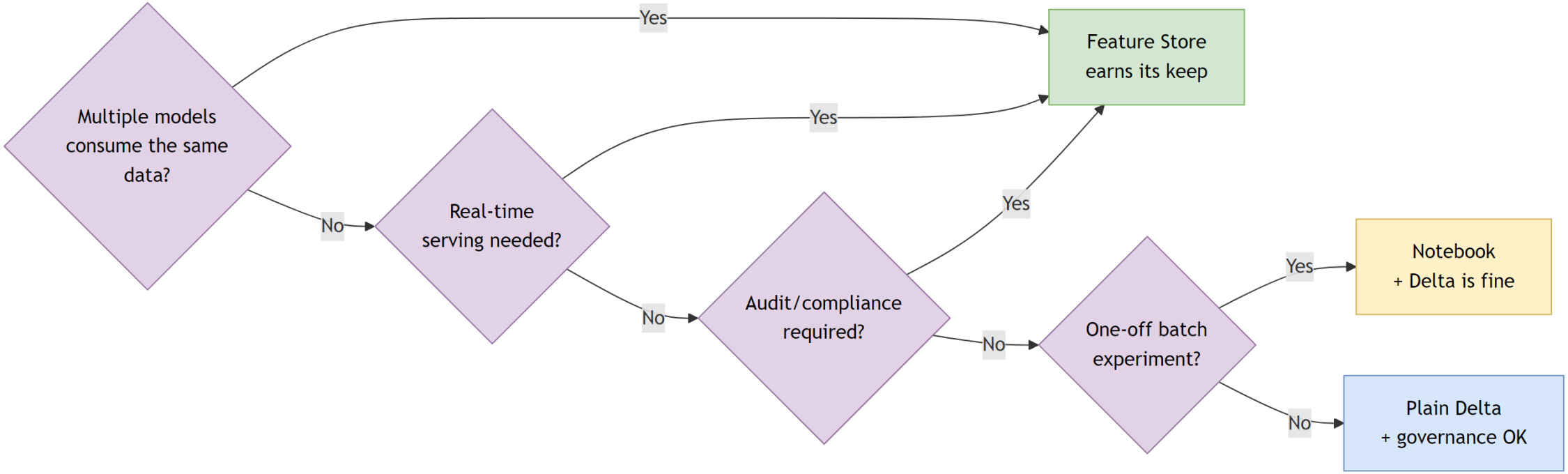
Per-FeatureSet SLOs (suggested defaults)

SLO	Target	Why
Materialization success rate	≥ 99 % over 30 days	Primary indicator of offline freshness
Materialization end-to-end latency	P95 ≤ 2× scheduled interval	A run longer than the cadence will pile up
Feature freshness	≤ scheduled interval + 15 min	The contract DS and consumers depend on
Schema conformance	100 %	Schema drift breaks training/serving parity

PRE-PROMOTION TEST GATE (CI BEFORE `STAGE = PRODUCTION`)

- **Point-in-time correctness** — regenerated training set matches the prior captured set
- **Null ratio** within `baseline ± 5 pp`
- **Entity cardinality** within `±10 %` of the 7-day rolling average
- **Distribution drift** — KS-statistic below threshold (e.g., 0.1)

Decision tree — feature store or plain Delta table?



Top 5 anti-patterns

A1

Two storage accounts with sync jobs

Fabric and AML each keep their tooling, two truths drift. Use one shared ADLS Gen2.

A2

Blob versioning on Delta containers

Conflicts with the Delta `_delta_log`. Breaks `OPTIMIZE` / `VACUUM`. Cost rises 2–10×.

A3

Online store for a batch-only model

Pays Redis 24/7 for nothing. Enable only with an active sync endpoint.

A4

Account keys / SAS tokens in notebooks

Security review will block production. Use Managed Identity.

A5

Production SLA on a Preview activity

Fabric DF AML activity is in Preview. Use AML pipelines for SLA-bound batches.

A6

Registering a Gold table without a data contract

Silent degradation propagates to every model. Validate before each materialization.

Production checklist (excerpt)

STORAGE

- HNS enabled, PE enabled, public access OFF
- CMK via Key Vault
- Soft delete on, **blob versioning OFF on Delta containers**
- Trusted Workspace Access for Fabric
- AML MI = `Storage Blob Data Contributor`

IDENTITY & CI/CD

- Workload identity federation in CI
- PR gates on `feature_sets/**`
- `az ml feature-set validate` on every PR
- Environment-promoted CD, no long-lived secrets

FEATURESET & MODEL

- Source data has timestamp + entity columns
- Online store **only** when sync endpoint exists
- Each model embeds its `feature_retrieval_spec.yaml`
- MLflow tracks every training run

GOVERNANCE

- Purview ingests Fabric + AML lineages
- Each model has owner, business case, monitoring plan
- Audit retention ≥ 5 years (EU AI Act high-risk)
- Quarterly access reviews
- **Data contract per Gold source enforced before materialization**

TAKE-AWAYS

One contract. Two views. Three layers.

ADLS GEN2 SHARED. AML OWNS THE CONTRACT. FABRIC AND AML BOTH RUN EXECUTION.

Build everything else on those four sentences.

References

- **Full guide** — `markdown/Fabric_AzureML_Feature_Store.md` (`pdf/`)
- **Microsoft Learn — Fabric × Azure ML batch endpoints** — `how-to-use-batch-fabric`
- **Microsoft Learn — AML Managed Feature Store** — `concept-what-is-managed-feature-store`
- **Microsoft Learn — OneLake datastore (preview)** — `how-to-datastore` — section Create a OneLake datastore (preview)
- **Tech Community — Build your feature engineering system on AML Managed FS and Microsoft Fabric (2024)** — techcommunity.microsoft.com
- **Repository** — github.com/fredgis/fabric-foundry-kb